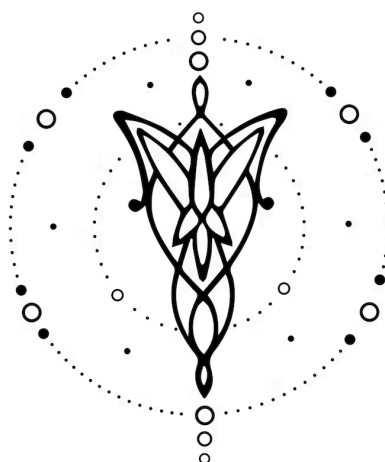




Especificación-Meeple

Diseño e Implementación de un Lenguaje

30 de Agosto de 2026

Indice:

1. Equipo	2
2. Repositorio	2
3. Dominio	2
3.1. Mapeo del dominio	2
3.2. Artefacto de salida	3
4. Construcciones	3
4.1. Elementos léxicos y tipos de datos	3
4.2. Componentes del juego	4
4.3. Expresiones y agregaciones	4
4.4. Control de flujo	5
4.5. Acciones del dominio	5
4.6. Decisiones y estrategias	6
4.7. Condiciones de fin	6
4.8. Simulación y reportes	7
5. Casos de prueba	7
5.1. Casos de prueba de aceptación	7
5.2. Casos de prueba de rechazo	8
6. Ejemplos	8
6.1. Un juego de carrera tipo “La Oca”	8
6.2. Un duelo de cartas con estrategias enfrentadas	9

1. Equipo

Nombres	Apellidos	Legajos	E-mails
Carolina	Goldbaum	65112	cgoldbaum@itba.edu.ar
Maria	Resek	65528	mresek@itba.edu.ar
Santiago	Fraga	64346	safraga@itba.edu.ar
Ivonne	Choe	64880	ichoe@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/cgoldbaum/meeple-compiler>

3. Dominio

Desarrollar un lenguaje que permita definir y simular juegos de mesa por turnos (*tabletop / board games*) con componentes básicos: jugadores, un tablero lineal, fichas, dados y cartas. El lenguaje debe permitir describir la configuración inicial de una partida, la lógica de cada turno, las políticas de decisión que siguen los jugadores y las condiciones de finalización, así como ejecutar la simulación masiva de partidas para obtener métricas agregadas.

Los jugadores son siempre agentes simulados que resuelven cada punto de decisión aplicando la estrategia que el programador les asignó.

La diferencia fundamental con un simulador convencional es que Meeple no solo describe cómo se juega, sino también cómo deciden los jugadores, permitiendo simular decenas de partidas bajo distintas políticas y observar métricas (duración promedio, tasa de victoria por estrategia) para descubrir qué política domina o si el juego está balanceado.

3.1. Mapeo del dominio

Segregando el grupo de conceptos, se mapean los sustantivos del dominio a tipos de datos, los verbos a acciones, y las preposiciones a dependencias de creación y pertenencia:

- **(Componentes (sustantivos → tipos):** `player`, `board`, `cell`, `piece`, `die`, `deck`, `strategy`, y los tipos nominales de carta declarados mediante `cardtype`.
- **Tipos clásicos:** `integer`, `boolean`, `string`.
- **Acciones:** `roll` (tirar), `place`(colocar), `move` (mover), `shuffle`(mezclar),`draw` (robar), `play` (jugar), `give / take` (dar / quitar), `ask`(consultar la decisión de un agente), `log` (registrar).
- **Preposiciones composicionales:** `in`, `on`, `to`, `from`, `of`, `per`, `by`, `where`, `uses`, que definen dependencias de creación, pertenencia y cuantificación (una ficha se coloca `on` una celda, un mazo es `of` un tipo de carta ya declarado, un mazo existe `per` `player`, un jugador `uses` una estrategia, etc.).

3.2. Artefacto de salida

El compilador de Meeple traduce un programa fuente en un programa en C que, una vez compilado y ejecutado, corre las partidas descritas. El programa generado ofrece un único punto de entrada no interactivo (simulate), que corre partidas masivas y emite un reporte de métricas por consola.

4. Construcciones

El lenguaje Meeple ofrece las siguientes construcciones, prestaciones y funcionalidades.

4.1. Elementos léxicos y tipos de datos

- Se admitirán comentarios de línea (`// ...`) y de bloque (`/* ... */`), que serán descartados por el analizador léxico. Se proveerán enteros con signo, booleanos (`true`, `false`) y cadenas entre comillas dobles.
- Se proveerán los tipos clásicos `integer`, `boolean` y `string`.
- Se proveerán los tipos de dominio `player`, `board`, `piece`, `dice` y `deck`.
- Se podrán declarar tipos nominales de carta mediante `cardtype`, indicando el nombre y el tipo de cada campo. Cada `cardtype` introduce un tipo estático nuevo. La palabra `card` se emplea únicamente para introducir el literal de una carta dentro de un mazo, y no es un tipo.

```
cardtype Naipe {  
    integer valor;  
    string palo;  
}
```

- Las variables podrán ser vectores de cualquiera de los tipos anteriores (e.j., `Naipe[ ]`, `player[ ]`), indexables mediante `[i]`.
- Se proveerá el literal `none`, que representa la ausencia de un valor de tipo referencia (carta o jugador), y que solo puede compararse mediante `==` y `!=`.
- Cada tipo del dominio expone un conjunto fijo de miembros predefinidos, verificables en tiempo de compilación:

Tipos	Miembros
<code>player</code>	<code>.name</code> , <code>.index</code> , <code>.score</code> , <code>.pieces</code> , <code>.strategy</code> , <code><mazo per player></code>
<code>piece</code>	<code>.owner</code> , <code>.cell</code>
<code>board</code>	<code>.size</code> , <code>.cell(i)</code>
<code>die</code>	<code>.min</code> , <code>.max</code> , <code>.faces</code>
<code>deck</code>	<code>.size</code> , <code>.cardtype</code>
<code><cardtype></code>	los campos declarados en ese <code>cardtype</code>

- Se proveerán las variables globales predefinidas `current` (jugador activo, `player`), `players` (`player[]`) y `turn_number` (`integer`). Un turno es la acción de un único jugador.

4.2. Componentes del juego

- Se podrá definir uno o varios juegos (`game`), cada uno con nombre y rango de jugadores (mínimo y máximo). Dentro de un `game`, toda entidad debe declararse antes de ser referenciada.
- Se podrá crear un único tablero (`board`) lineal por juego, indicando su cantidad de celdas, numeradas de 0 a `n-1`.

`board Tablero cells 63;`

- Se podrán crear fichas o piezas, pueden crearse individualmente o asignarse a cada jugador con `per player`. En este último caso, cada pieza queda asociada a su jugador.

`Piece token per player;`

- Se podrán crear dados (`die`) con un rango de caras o con una serie de valores.

**`die D6 faces 1 to 6;`
`die Moneda faces {0, 1};`**

- Se pueden crear mazos (`deck`) a partir de un `cardtype` ya definido. Estos mazos pueden ser generales o pertenecer a cada jugador usando `per player` funcionando como su mano de cartas. Las cartas del mazo deben respetar exactamente los campos definidos en el `cardtype`.

```
deck Mazo of Naipe {
  card{
    valor:1;
    Palo: "oro";
  }
}
deck Mano of Naipe per player;
```

4.3. Expresiones y agregaciones

- Se proveerán operadores relacionales: `<`, `>`, `==`, `!=`, `<=` y `>=`.
- Se proveerán operaciones aritméticas básicas: `+`, `-`, `*`, `/` y `%`.
- Se proveerán operaciones lógicas básicas: `and`, `or` y `not`.
- Se proveerán agregaciones sobre colecciones del dominio, parentizadas y con la misma forma de ligadura de variable. La cláusula `where` es opcional; `max` y `min` devuelven `none` sobre una colección vacía.

Agregaciones	Resultados
<code>count(c in Mazo)</code>	<code>integer</code>

Agregaciones	Resultados
count(c in Mazo where c.palo == "oro")	integer
sum(p in players by p.score)	integer
max(p in players by p.score)	player
min(c in current.Mano by c.valor)	Naipes

```
// Elegir la carta jugable de mayor valor:
Naipes[ ] jugables = select(c in current.mano where c.valor <=
current.oro);
Naipes mejor = max(c in jugables by c.valor); //si no hay ninguna
```

4.4. Control de flujo

- Se proveerán estructuras de control if-else, for, while y repeat n.
- Se podrá definir un bloque setup que describa la configuración inicial de la partida.
- Se podrá definir un bloque turn que describa la lógica de cada turno, con acceso al jugador activo mediante current.
- Se podrá declarar el orden de juego (turn order clockwise o counterclockwise).

4.5. Acciones del dominio

Se proveerán las siguientes acciones, donde n denota una expresión entera cualquiera:

Acciones	Descripciones
roll D6	Tira un dado; devuelve un integer.
move p.token forward n	Avanza n celdas sobre el tablero lineal.
shuffle Mazo	Mezcla un mazo.
draw n from Mazo to p.Mano	Transfiere n cartas entre mazos del mismo cardtype.
play c from current.Mano to Mesa	Juega una carta específica a otro mazo.
give n to p	Suma n al score del jugador p.
take n from p	Resta n al score del jugador p.
log "...{1}...{2}", a, b	Emite un mensaje con sustitución posicional.

- Si el mazo de origen de un draw contiene menos de n cartas, se transfieren todas las disponibles y la acción no falla. Si la carta indicada en un play no pertenece al mazo de origen, la acción no tiene efecto.

- La cantidad y el tipo de los argumentos de log deberán corresponderse con los marcadores posicionales presentes en la cadena de formato; la discrepancia es un error semántico.

4.6. Decisiones y estrategias

- Se podrán declarar puntos de decisión (decision), indicando el tipo del conjunto de opciones y el tipo del resultado. Ambos tipos deben estar previamente declarados.

```
decision jugar(opciones: Naipe[ ]) -> Naipe;
```

- Se podrán definir estrategias (strategy) que resuelven cada punto de decisión declarado mediante una política: prefer max <expr>, prefer min <expr>, prefer random o prefer first.

```
strategy Agresivo { jugar: prefer max option.valor; }  
strategy Conservador { jugar: prefer min option.valor; }  
strategy Azar { jugar: prefer random; }
```

- Dentro del cuerpo de una strategy estarán en alcance la variable predefinida option (el elemento del conjunto de opciones que se está evaluando), current (el jugador que decide), players y los mazos del juego. No estarán en alcance las variables locales del turn: una estrategia decide únicamente en función del estado del juego y de la opción evaluada.
- Se podrá asignar una estrategia a un jugador mediante uses. Un jugador que no reciba asignación explícita resolverá todos sus puntos de decisión con la política prefer first, es decir, eligiendo el primer elemento del conjunto de opciones.

```
players[0] uses Agresivo;  
players[1] uses Conservador;
```

- Se podrá consultar la decisión de un agente mediante ask, que devuelve el elemento elegido, o none si el conjunto de opciones está vacío.

```
Naipe elegida = ask current for jugar (  
  select(c in current.Mano where c.valor <= current.score));
```

- Toda estrategia deberá resolver todos los puntos de decisión declarados en el juego; la omisión de alguno es un error semántico.

4.7. Condiciones de fin

Se proveerán dos formas de terminación:

Formas	Semánticas
<code>win when <cond>;</code>	Declarativa. Se evalúa al final de cada turno sobre <code>current</code> ; si es verdadera, <code>current</code> gana.
<code>end after <n> turns;</code>	Cota dura sobre <code>turn_number</code> ; termina sin ganador al alcanzarla.

Todo juego deberá declarar al menos una condición `win when`. La sentencia `end after` es opcional y actúa como cota de seguridad para evitar simulaciones no terminantes; en su ausencia, el runtime impone una cota fija por defecto.

4.8. Simulación y reportes

- Se podrá fijar la semilla del generador pseudoaleatorio (seed `n`), garantizando que toda simulación sea reproducible. En ausencia de una declaración explícita se utiliza una semilla fija por defecto, de modo que la reproducibilidad nunca depende del reloj del sistema.
- Se podrá simular una o varias partidas indicando el juego a simular y la cantidad de jugadores. El nombre indicado deberá corresponder a un game previamente declarado; referenciar un juego inexistente es un error semántico. La cantidad de jugadores deberá estar dentro del rango declarado por ese game; excederlo o no alcanzarlo es también un error semántico. Las palabras `game` y `games` son intercambiables.

```
simulate 1000 games of "La Oca" with 4 players;
```

- Se podrán emitir métricas agregadas sobre la simulación anterior de manera inmediata utilizando `report`, siendo la salida por consola.

```
report {
  winrate by player;
  winrate by strategy;
  avg turns;
}
```

- Se podrá regular el nivel de detalle de la traza mediante `log level verbose | none`, de modo que una simulación masiva no emita una línea por acción. El valor por defecto es `none`.

5. Casos de prueba

5.1. Casos de prueba de aceptación

Se proponen los siguientes casos iniciales de prueba de aceptación:

- Un programa que defina un juego mínimo con nombre y rango de jugadores.
- Un programa que construya un tablero lineal de `N` celdas.

3. Un programa que declare un dado con un rango de caras y otro con un conjunto explícito de valores.
4. Un programa que declare un **cardtype** y un mazo de cartas conforme a él.
5. Un programa que declare un mazo general y un mazo **per player**.
6. Un programa con un bloque setup que coloque una ficha de cada jugador en la celda inicial.
7. Un programa donde turn tire un dado y mueva la ficha del jugador activo con `move ... forward`.
8. Un programa que use **if-else** dentro de un turno.
9. Un programa que itere con **for** sobre todos los jugadores.
10. Un programa que use **while** o **repeat** para repetir una acción.
11. Un programa que combine las cuatro operaciones aritméticas y el módulo.
12. Un programa que reparte cartas (**draw**) de un mazo a la mano de un jugador.
13. Un programa que use agregaciones (**count**, **sum**, **max**, **min**), con y sin cláusula **where**.
14. Un programa que compare un valor contra **none**.
15. Un programa que emite un **log** con marcadores posicionales y argumentos de tipos distintos.
16. Un programa que declare un punto de decisión y dos estrategias que lo resuelvan.
17. Un programa que asigne estrategias distintas a dos jugadores y consulte su decisión mediante **ask**.
18. Un programa que defina una condición de victoria con **win when**.
19. Un programa que termina sin ganador con **end after n turns**.
20. Un programa que fije la semilla y simule N partidas de un juego con K jugadores, emitiendo un **report**.

5.2. Casos de prueba de rechazo

Además, se proponen los siguientes casos de prueba de rechazo:

1. Un programa malformado (por ejemplo, falta un `;` o una llave sin cerrar).
2. Un programa que mueva o referencie una ficha o jugador que no fue declarado.
3. Un programa que declare un rango de jugadores inválido (mínimo mayor que máximo).
4. Un programa que opere entre tipos incompatibles (e.g., sumar una carta con un **integer**, o comparar un **player** con un **string**).
5. Un programa cuyo mazo contenga una carta con un campo no declarado en su **cardtype**.
6. Un programa que transfiera cartas con **draw** entre dos mazos de **cardtype** distinto.
7. Un programa cuyo log tenga más marcadores posicionales que argumentos.
8. Un programa con una **strategy** que no resuelva todos los puntos de decisión declarados.
9. Un programa que consulte con **ask** un punto de decisión no declarado.
10. Un programa que simule un juego que no fue declarado, o con más jugadores que el máximo permitido.
11. Un programa que no declare ninguna condición **win when**.

6. Ejemplos

6.1. Un juego de carrera tipo “La Oca”

Se crea un juego para 2 a 4 jugadores sobre un tablero lineal de 63 celdas. En cada turno el jugador activo tira un dado de 6 caras y avanza su ficha; el primero en alcanzar la última celda gana.

```
seed 42;
game "La Oca" {
```



```

players 2 to 4;
board Tablero cells 63;
die D6 faces 1 to 6;
piece token per player;

setup {
  for (p in players) {
    place p.token on Tablero.cell(0);
  }
}

turn {
  integer pasos = roll D6;
  move current.token forward pasos;
  log "{1} avanza {2} y quedó en la celda {3}",
    current, pasos, current.token.cell;
}

win when current.token.cell >= 62;
end after 500 turns;
}

simulate 1000 games of "La Oca" with 4 players;
report { winrate by player; avg turns; }

```

Código 6.1: Un juego de carrera (simulación de balance). El setup recorre los jugadores con un for y coloca la ficha de cada uno en la celda inicial. El turn usa roll (que devuelve integer), move ... forward y log. win when define la condición de victoria y end after actúa como cota de seguridad. report emite la tasa de victoria por posición inicial y la duración promedio.

6.2. Un duelo de cartas con estrategias enfrentadas

Consiste en dos jugadores con políticas de decisión distintas compitiendo, para medir cuál domina.

```

seed 1;
game "Duelo de Cartas" {
  players 2 to 2;
  cardtype Carta { integer ataque; integer costo; }
  deck Mazo of Carta {
    card { ataque: 1; costo: 1; } card { ataque: 2; costo: 1; }
    card { ataque: 3; costo: 2; } card { ataque: 4; costo: 3; }
    card { ataque: 5; costo: 4; } card { ataque: 6; costo: 4; }
    card { ataque: 7; costo: 5; } card { ataque: 9; costo: 7; }
  }
  deck Mano of Carta per player;
  deck Mesa of Carta;

  decision jugar(opciones: Carta[]) -> Carta;
  strategy Agresivo { jugar: prefer max option.ataque; }
}

```

```
strategy Economico { jugar: prefer min option.costo; }

setup {
  shuffle Mazo;
  for (p in players) {
    draw 3 from Mazo to p.Mano;
    give 5 to p;
  }
  players[0] uses Agresivo;
  players[1] uses Economico;
}

turn {
  if (count(c in Mazo) > 0) {
    draw 1 from Mazo to current.Mano;
  }
  Carta elegida = ask current for jugar(
    current.Mano);
  if (elegida != none and elegida.costo <= current.score) {
    take elegida.costo from current;
    play elegida from current.Mano to Mesa;
    give elegida.ataque to current;
  }
}

win when current.score >= 15;
end after 20 turns;
}

simulate 5000 games of "Duelo de Cartas" with 2 players;
report { winrate by strategy; avg turns; }
```

Código 6.2: El punto de decisión se declara sobre el cardtype concreto (Carta[]), lo que permite que prefer max option.ataque type. ask ... for ... consulta al agente y devuelve none si no hay opciones jugables. El report ... by strategy responde si existe una estrategia dominante.